Project Aurora

Kriti 2026

Documentation

Team No. 2617

Contents

1. Abstract

SpeakX Aurora is a fully working, domain-agnostic, self-learning notification and communication orchestrator. It combines MECE behavioural segmentation, Octalysis game-psychology driven bilingual LLM content generation, and an automated feedback loop to deliver the right message, to the right user, at the right time — and measurably improves across iterations. The system is built for SpeakX (Kriti 2026 Problem Statement) but requires only a configuration swap to operate on any B2C or B2B domain.

2. Introduction

2.1. Context

Across EdTech, FinTech, and SaaS, user communication is the single most powerful growth lever, yet today's notification systems are rule-based, segment-blind, and timing-agnostic. SpeakX is an AI-powered English speaking and communication learning app targeting Hindi and regional-language speakers in Tier 1, 2, and 3 cities across India (age group 20–45). Core users are working professionals, college students, and homemakers seeking better jobs, promotions, and social confidence.

2.2. Problem Statement

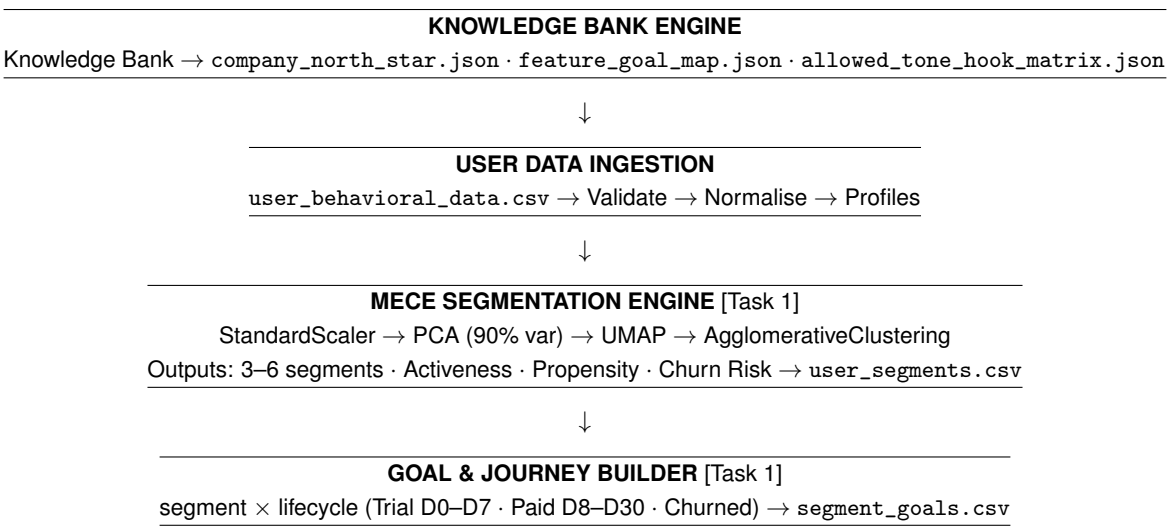
Design and build a fully working, self-learning notification orchestrator that demonstrates how user engagement can become smarter, more personalised, and continuously improving. The solution must combine data intelligence, AI-powered content generation, and behavioural learning across three integrated tasks. The architecture must be domain-agnostic: swapping the Knowledge Bank and user data keeps the orchestrator core unchanged.

2.3. Design Principles

- **MECE** — every user belongs to exactly one segment; no user is left unaddressed.
- **Octalysis** — all notification content is grounded in the eight core human motivational drives.
- **Domain-agnostic** — `brand_config.json` is the only coupling between business logic and data schema.
- **Auditable** — every Iteration 0 to Iteration 1 change is logged with causal reasoning in `learning_delta_report.csv`.

3. System Architecture

The pipeline is a linear directed graph of six processing stages. Experiment results provided by SpeakX on Demo Day are injected as a side-channel into the Classification + Learning Engine, closing the loop from Iteration 0 to Iteration 1.



↓

THEME ENGINE + TEMPLATE GENERATOR [Task 2]
features → Octalysis Drive → 5 templates × Segment × Lifecycle
Bilingual EN + HI · Groq LLaMA-3.3-70B · Tone guardrails
Outputs: `communication_themes.csv` · `message_templates.csv`

↓

TIMING + FREQUENCY OPTIMIZER [Task 2]
`notif_open_rate_30d` · user count per timing window per segment → 6 windows · activeness → 3/5/7–9 notifs/day
Output: `timing_recommendations.csv` · `user_notification_schedule.csv`

↓ *(experiment_results.csv injected here on Demo Day)* ↓

CLASSIFICATION + LEARNING ENGINE [Task 3]
GOOD / NEUTRAL / BAD → Suppress · Promote as LLM context
Re-run pipeline → Iteration 1 outputs + `learning_delta_report.csv`

4. Task 1 — System Architecture & Intelligenc

4.1. Knowledge Bank Engine

When the evaluator provides a company Knowledge Bank (plain text, Markdown, or PDF), the KB Engine automatically extracts all structured intelligence required by the orchestrator using an LLM-powered parser. No manual configuration is needed; the extraction produces valid JSON outputs directly.

4.1.1. Accepted Input Formats

- Plain text (`.txt`) or Markdown (`.md`) — pasted or uploaded directly into the notebook cell
- PDF — converted to text via `pdfplumber` before parsing
- Structured prompt injection — evaluator can also supply the KB as a structured prompt string

4.1.2. Extraction Pipeline

The KB text is passed to `llama-3.3-70b-versatile` via Groq with a structured extraction prompt. The LLM is instructed to return valid JSON only. The output is parsed and written to three files:

1. `company_north_star.json` — auto-inferred North Star metric name, definition, measurable proxy, and threshold
2. `feature_goal_map.json` — each product feature mapped to its `db_col`, `inferred_goal`, `octalysis_drive`, and causal reasoning
3. `allowed_tone_hook_matrix.json` — allowed tones, disallowed tones, and full Octalysis hook taxonomy derived from brand voice guidelines

The extracted `feature_goal_map.json` drives the entire downstream pipeline. Swapping the input KB document is the only step required to redeploy the orchestrator for a new domain.

4.2. User Data Ingestion

Accepts `user_behavioral_data.csv` per the schema in Section 7. Validates column presence and types, handles missing data, applies `StandardScaler` normalisation across all numeric features, and outputs clean user profiles.

4.3. MECE Segmentation Engine

StandardScaler → PCA (90% variance retained) → UMAP ($n\_components = 2$) → AgglomerativeClustering (Ward linkage, $K = 2 \dots 8$)

Silhouette scoring selects the optimal K, producing 3–6 MECE segments. Each segment receives three composite scores via sigmoid-normalised Z-score aggregation with direction-based weights (positive features weighted $+1$, inverse-risk features weighted -1):

- **Activeness Score** — `sessions_last_7d`, `exercises_completed_7d`, `streak_current`, `notif_open_rate_30d`
- **Propensity Score** — `streak_current`, `coins_balance`, `feature_leaderboard_viewed`, `feature_ai_tutor_used`, `motivation_score`
- **Churn Risk Score** — `lifecycle_stage_inactive`, `lifecycle_stage_churned`, `notif_open_rate_30d`, `motivation_score`

4.4. Goal & Journey Builder

Reads `brand_config.json` and `user_segments.csv` to produce `segment_goals.csv` with day-on-day sub-goals per segment $\times$ lifecycle stage. Lifecycle stages and primary goals:

Stage	Days	Primary Goal
Trial	D0–D7	Conversion
Paid	D8–D30	Retention
Inactive	D30+	Reactivation
Churned	D60+	Win-back

4.5. Task 1 Output Files

File	Description
`company_north_star.json`	Inferred North Star, justification, measurable proxy
`feature_goal_map.json`	Feature $\rightarrow$ db_col $\rightarrow$ lifecycle $\rightarrow$ outcome mapping
`allowed_tone_hook_matrix.json`	Allowed/disallowed tones, full Octalysis hook taxonomy
`user_segments.csv`	MECE segments with Propensity, Activeness, Churn Risk
`segment_goals.csv`	Goals, sub-goals, day-on-day focus per segment $\times$ lifecycle

5. Task 2 — Communication & Timing Intelligence

5.1. Theme Engine

Iterates `feature_goal_map.json` to build `FEATURE_TO_DRIVE` (features $\rightarrow$ Octalysis drive). Using the segmentation deviation matrix, each segment is assigned: `primary_theme` (highest positive deviation feature's drive), `secondary_theme` (next distinct positive deviation), and `avoid_theme` (highest negative deviation). Outputs `communication_themes.csv`.

5.2. Octalysis 8 Core Drives

#	Drive	Hook Directive
1	Epic Meaning	User is part of a grand mission, an elite group
2	Accomplishment	Levelling up, hitting a milestone, overcoming a challenge
3	Empowerment	Discovering a smarter strategy, unlocking a new tool
4	Ownership	Accumulating rewards, claiming virtual assets
5	Social Influence	Peer comparison, climbing ranks, defending status
6	Scarcity	Exclusive access, limited-time windows
7	Unpredictability	Mystery reward, cliffhanger that triggers an open
8	Loss Avoidance	Streak at risk, progress about to expire

5.3. Template Generator

- **Model:** `llama-3.3-70b-versatile` via Groq API
- **Volume:** exactly 5 templates per Segment $\times$ Lifecycle combination
- **Narrative arc:** T1_Seed (low intensity, early day) $\rightarrow$ T2_Tease (build anticipation) $\rightarrow$ T3_Anchor (preferred hour, strongest hook) $\rightarrow$ T4_Welcome (in-app reinforcement) $\rightarrow$ T5_WrapUp (end-of-day close)
- **Bilingual:** every template carries `content_english` and `content_hindi`
- **Placeholders:** `{first_name}`, `{streak_current}`, `{coins_balance}`, `{feature_name}`
- Tone guardrails from `brand_config["guardrails"]` injected into every LLM call

5.4. Timing Optimizer

For each segment, the optimizer computes a crosstab of `notif_open_rate_30d` (mean open rate) and user count per timing window. Windows are ranked by a combined score that weights both average open rate and the volume of users active in that window, ensuring that the top-3 recommended windows reflect both engagement signal strength and statistical reliability. Low-population windows are penalised even if their mean open rate is high, preventing spurious recommendations driven by small sample sizes. The output is written to `timing_recommendations.csv` with the top-3 window ranking per segment alongside expected open-rate lift.

The timing windows used for optimization are defined as follows:

Window	Time Range	Typical Use
`early_morning`	06:00–08:59	Morning motivation, habit trigger
`mid_morning`	09:00–11:59	Work break reminder
`afternoon`	12:00–14:59	Lunch break engagement
`late_afternoon`	15:00–17:59	Productivity boost
`evening`	18:00–20:59	Post-work learning
`night`	21:00–23:59	End-of-day recap, streak save

5.5. Frequency Optimizer

Activeness Score	Notifs/day	Churn Risk	Strategy
> 0.7 (Highly Active)	7–9	Low	Maximum engagement
0.4–0.7 (Moderate)	5–6	Medium	Balanced approach
< 0.4 (Low Active)	3–4	High	Conservative, avoid fatigue

Guardrail override: if segment `uninstall_rate > 2%`, reduce notification frequency by 2/day regardless of activeness score.

Each user's schedule anchors on `preferred_hour` with fixed offsets (T$-$8h, T$-$4h, T$-$2h, T$+$0, T$+$1m, T$+$15m, T$+$30m, T$+$45m, T$+$2h). Every `notif_X` slot carries `{template_id, time, channel}`.

5.6. Task 2 Output Files

File	Description
`communication_themes.csv`	Primary/secondary themes, tone, hooks per segment
`message_templates.csv`	5 bilingual templates per combination (EN + HI)
`timing_recommendations.csv`	Optimal windows per segment with expected CTR
`user_notification_schedule.csv`	User-wise daily schedule, `notif_1` through `notif_9`

6. Task 3 — Execution & Self-Learning

Ingests `experiment_results.csv` provided by SpeakX on Demo Day and closes the learning loop between Iteration 0 and Iteration 1.

6.1. Classification Engine

Class	CTR	Engagement Rate	Action
GOOD	$> 15\%$	$> 40\%$	Promote, use as few-shot reference
NEUTRAL	5–15%	20–40%	Iterate, A/B test further
BAD	$< 5\%$	$< 20\%$	Suppress, do not use

6.2. Learning Engine

1. **Suppress** BAD templates — removed from all Iteration 1 schedules.
2. **Promote** GOOD templates as few-shot reference context injected into the LLM system prompt for regeneration. The optimal timing window identified from GOOD template performance is learned per segment; Iteration 1 schedules are anchored to this learned window. A/B testing is conducted by splitting users within each segment into control (Iteration 0 window) and treatment (learned optimal window) groups, with CTR and engagement rate compared across groups to validate the window shift before full rollout.
3. **Update timing** — if a segment's GOOD templates concentrate in a specific window, `timing_recommendations.csv` is updated in Iteration 1.
4. **Adjust frequency** — segments where `uninstall_rate > 2%` have daily notification count reduced by 2.

6.3. Delta Reporter

Every change between Iteration 0 and Iteration 1 is logged to `learning_delta_report.csv`:

Column	Type	Description
`entity_type`	enum	`template` · `segment` · `timing` · `frequency` · `schedule`
`entity_id`	string	Specific template_id, segment_id, etc.
`change_type`	enum	`SUPPRESSED` · `PROMOTED` · `WINDOW_SHIFT` · `FREQUENCY_REDUCED` · `REGENERATED`
`metric_trigger`	float	Exact metric value that caused the change
`before_value`	mixed	Value in Iteration 0
`after_value`	mixed	Value in Iteration 1
`explanation`	string	Full human-readable causal sentence

6.4. Task 3 Output Files

File	Description
`iteration_1_after_learning/message_templates.csv`	Improved templates post-learning
`iteration_1_after_learning/timing_recommendations.csv`	Updated timing windows
`iteration_1_after_learning/user_notification_schedule.csv`	Revised schedules
`learning_delta_report.csv`	All changes with causal reasoning

7. Data Schemas

7.1. Input: `user_behavioral_data.csv`

Column	Type	Description
`user_id`	string	Unique anonymised identifier
`lifecycle_stage`	enum	`trial` · `paid` · `churned` · `inactive`
`days_since_signup`	int	Days since registration
`age_band`	string	`18-24` · `25-34` · `35-44` · `45+`
`region`	string	`tier1` · `tier2` · `tier3`
`sessions_last_7d`	int	Sessions in last 7 days
`exercises_completed_7d`	int	Exercises completed in last 7 days
`streak_current`	int	Current active streak in days
`coins_balance`	int	Gamification coins held
`feature_ai_tutor_used`	bool	AI tutor engagement flag
`feature_leaderboard_viewed`	bool	Leaderboard engagement flag
`preferred_hour`	int	Most active hour (0–23)
`notif_open_rate_30d`	float	30-day notification open rate (0–1)
`motivation_score`	float	Inferred motivation score (0–1)

7.2. Experiment Results: `experiment_results.csv`

Column	Type	Description
`template_id`	string	Unique template identifier
`segment_id`	string	Target segment ID
`lifecycle_stage`	string	User lifecycle stage
`goal`	string	Template goal alignment
`theme`	string	Octalysis hook used
`notification_window`	string	Time window of delivery
`total_sends`	integer	Number of notifications sent
`total_opens`	integer	Number of notifications opened
`total_engagements`	integer	Number of in-app actions taken
`ctr`	float	Click-through rate (opens / sends)
`engagement_rate`	float	Engagement rate (engagements / opens)
`uninstall_rate`	float	Uninstall rate post-notification
`performance_status`	string	Template classification (GOOD/BAD/NEUTRAL)

8. Evaluation Criteria

As defined in the Kriti 2026 Mid Prep Problem Statement:

Dimension	Weight	Description
System Completeness	15%	All files present, correct schemas, runnable codebase
Segmentation Quality	15%	MECE compliance, meaningful descriptions, differentiated propensities
Messaging Intelligence	25%	Template variety, theme relevance, bilingual quality, hook diversity
Timing & Frequency	10%	Segment-aware frequency (3–9/day), timing optimisation
Learning & Evolution	25%	Observable delta, causal explanations, real improvement
Extensibility	5%	Domain-agnostic architecture, configurable for industries
Presentation	5%	Overall clarity, professionalism, demo quality
Total	**100%**	

8.1. Non-Negotiable Constraints

- Fully working end-to-end system with complete runnable codebase.
- Demonstrate real Iteration 0 $\rightarrow$ Iteration 1 improvement with measurable delta.
- Export all outputs in strict CSV/JSON formats per defined schemas.

AUTO-FAIL triggers: hardcoded outputs, mock learning, presentations without exports, no delta between iterations, missing files.

9. Directory Structure

```
SpeakX_AURORA_<2617>.zip
|-- iteration_0_before_learning/
|   |-- company_north_star.json
|   |-- feature_goal_map.json
|   `-- user_notification_schedule.csv
|-- iteration_1_after_learning/
|   `-- user_notification_schedule.csv
|-- experiment_results.csv
|-- learning_delta_report.csv
|-- codebase/
|   |-- task1_system_architecture.ipynb
|   `-- brand_config.json
`-- README.txt
```

10. Setup & Execution

10.1. Execution Order

1. `task1_system_architecture.ipynb` — run all cells.
2. `task2_communication_intelligence.ipynb` — run all cells.
3. `task3_execution_self_learning.ipynb` — run after experiment results.